

Informe Técnico: Sistema de Recuperación de Información

Autor: David Esteban Morales

Materia: Recuperación de Información — Prof. Iván Carrera

Fecha: 01 de junio de 2026

1. Descripción del Corpus

Se utilizó el dataset **Reuters “Uncovering Financial Insights with the Reuters 2”** de Kaggle (`ModApte_train.csv`), que contiene aproximadamente **9,000 documentos de noticias financieras** (mercados, commodities, política monetaria, comercio internacional).

Para cada documento se concatenan los campos `title` y `text` como entrada al sistema. La columna `topics` (categorías como *earn*, *acq*, *grain*, *trade*) se emplea para construir los juicios de relevancia (`qrels`) en la evaluación automática.

2. Arquitectura del Sistema

2.1 Estructura del proyecto

```
proyecto/  
  models/  
    base.py          # Clase abstracta RetrievalModel  
    jaccard.py       # Similitud Jaccard  
    tfidf.py         # TF-IDF con coseno  
    bm25.py          # BM25 (Okapi)
```

<code>semantic.py</code>	<code># Embeddings + FAISS</code>
<code>preprocessing.py</code>	<code># Pipeline de texto + caché + índice invertido</code>
<code>evaluation.py</code>	<code># Evaluación batch sobre qrels</code>
<code>metrics.py</code>	<code># P@K, R@K, AP, MAP (implementación propia)</code>
<code>qrels.py</code>	<code># Extracción de qrels desde topics de Reuters</code>
<code>main.py</code>	<code># CLI interactiva con Rich</code>

Todos los modelos heredan de `RetrievalModel` (clase abstracta con `@abstractmethod search()`), lo que garantiza una interfaz uniforme y permite ejecutarlos polimórficamente desde la CLI.

2.2 Preprocesamiento (`preprocessing.py`)

El pipeline de texto aplica secuencialmente:

1. **Limpieza regex** — elimina todo carácter no alfabético
2. **Tokenización** — NLTK `word_tokenize`
3. **Remoción de stopwords** — lista estándar de inglés (NLTK)
4. **Stemming** — `SnowballStemmer('english')`

El método `load_or_build()` implementa **caché con joblib**: en la primera ejecución procesa el corpus completo y serializa el `DataFrame` y el índice invertido en disco (`cache/`). En ejecuciones posteriores carga directamente desde caché.

2.3 Índice Invertido

Estructura: `{término: {doc_id: frecuencia}}` construido con `defaultdict` y `Counter`. Los tres modelos clásicos (Jaccard, TF-IDF, BM25) operan directamente sobre este índice — no se usa ninguna librería externa para la recuperación.

2.4 Modelos de Recuperación

Jaccard (JaccardModel)

Calcula la similitud de conjuntos entre los términos de la query y los de cada documento:

$$Jaccard(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

- Opera de forma **binaria**: solo importa si el término está presente, no su frecuencia.
- Precomputa **doc_unique_terms** (cardinalidad del vocabulario de cada documento) en **__init__** para evitar recalcular en cada query.
- Recorre únicamente los posting lists de los términos de la query en el índice invertido.

TF-IDF Coseno (TFIDFModel)

Esquema de pesos **Inc.ltc** (notación SMART):

- **TF**: logarítmico $\rightarrow 1 + \log(\text{tf})$
- **IDF**: clásico $\rightarrow \log(N / \text{df}_t)$
- **Normalización**: coseno $\rightarrow$ divide el producto punto por $\|\text{doc}\| \times \|\text{query}\|$

Las normas de documentos se **precomputan en __init__** usando todos los términos del corpus (no solo los de la query), lo cual es fundamental para obtener scores diferenciados.

BM25 (BM25Model)

$$\text{score}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{\text{tf}(t, D) \cdot (k_1 + 1)}{\text{tf}(t, D) + k_1 \cdot (1 - b + b \cdot \frac{dl}{\text{avgdl}})}$$

- **Parámetros**: $k_1=1.5$, $b=0.75$ (estándar de la literatura)
- **Diferencia clave con TF-IDF**: penalización explícita por longitud del documento (dl/avgdl) integrada en la fórmula; no necesita normalización coseno.
- Precomputa **doc_lengths** y **avgdl** en la inicialización.

Semántico (SemanticModel)

Utiliza **sentence-transformers** con el modelo **all-MiniLM-L6-v2** (384 dimensiones) para generar embeddings densos de cada documento.

- **Base vectorial:** FAISS con índice **IndexFlatIP** (inner product). Los vectores se normalizan con L2 antes de insertarlos, por lo que el inner product equivale a **similitud coseno**.
- **Persistencia:** el índice FAISS se serializa en disco (**faiss_index.bin**) con **faiss.write_index** / **faiss.read_index**, evitando regenerar embeddings (~9000 docs) en cada ejecución.
- **Query sin preprocesar:** a diferencia de los modelos clásicos, la query se pasa en texto crudo al encoder — el modelo de lenguaje ya captura semántica sin necesidad de stemming.
- **Ventaja principal:** captura relaciones de significado (sinónimos, paráfrasis) que los modelos basados en términos no pueden detectar.

3. Evaluación

3.1 Qrels

Se extraen de la columna **topics** del CSV. Cada topic se usa como query; los documentos etiquetados con ese topic son los relevantes. Se filtran topics con menos de 10 documentos → **61 queries de evaluación**.

3.2 Métricas (implementación propia)

Métrica	Definición
Precision@K	Proporción de relevantes en el top-K
Recall@K	Proporción del total de relevantes que aparecen en el top-K
Average Precision	Premia que los relevantes estén en posiciones altas del ranking
MAP	Promedio de AP sobre las 61 queries

3.3 Resultados (k=10, 61 queries)

Modelo	MAP	avg P@10	avg R@10
Jaccard	0.0541	0.2246	0.0628
TF-IDF Coseno	0.0632	0.2557	0.0723
BM25	0.0666	0.2525	0.0742
Semántico	0.0801	0.3115	0.0861

Orden: Jaccard < TF-IDF < BM25 < Semántico (consistente con la literatura)

El recall es bajo (~6-8%) porque topics como “earn” tienen 2,877 docs relevantes y K=10.

4. Análisis Comparativo por Tipo de Consulta

Query 1: “earn” (término único, alta frecuencia)

[CAPTURA: resultados de los 4 modelos para “earn”]

Todos los modelos rinden de forma similar — no hay ambigüedad léxica ni ventaja semántica.

Query 2: “oil prices rising” (multi-token)

[CAPTURA: resultados de los 4 modelos para “oil prices rising”]

BM25 supera a TF-IDF gracias a la penalización por longitud, evitando que documentos largos dominen por acumular términos frecuentes.

Query 3: “chocolate commerce” (sinónimos)

[CAPTURA: resultados de los 4 modelos para “chocolate commerce”]

El modelo semántico destaca significativamente. Los modelos clásicos no recuperan documentos sobre “cocoa” porque el stemming no relaciona “chocolate” con “cocoa”. Los embeddings sí capturan esta equivalencia semántica.

Query 4: “japanese central bank interest rate policy” (query larga)

[CAPTURA: resultados de los 4 modelos para query larga]

Jaccard se diluye porque $|A \cap B|$ crece con la longitud de la query. BM25 pondera cada término independientemente y maneja mejor este caso.

5. Conclusiones

1. El orden de rendimiento **Jaccard** < **TF-IDF** < **BM25** < **Semántico** es consistente con la teoría de RI.
2. BM25 es el mejor modelo clásico gracias a su normalización por longitud y saturación de frecuencia.
3. La recuperación semántica (FAISS + all-MiniLM-L6-v2) muestra su mayor ventaja en queries con **variación léxica** donde los modelos basados en términos fallan por no capturar sinónimos.
4. Los modelos clásicos siguen siendo competitivos para queries literales sin gap semántico.
5. MAP es la métrica más informativa dado que la distribución de documentos relevantes por topic es muy desigual.